

The Relational Model

Author: Diane Horton



Learning outcomes

By the end of this unit, you will be able to

- Read and interpret a schema written in relational notation
- Understand the difference between schema and instance
- Identify instances that do and don't violate it
- Know how to use terminology, e.g., tuple, arity
- Explain and use the concepts key, superkey, and foreign key

Recap

- The relational model is based on the concept of a **relation** or **table**, such as these:

Teams	name	homeField	coach
	Rangers	Runnymede CI	Tarvo Sinervo
	Ducks	Humber Public	Tracy Zheng
	Choppers	High Park	Ammar Jalali

Games	homeTeam	awayTeam	homeGoals	awayGoals
	Rangers	Ducks	3	0
	Ducks	Choppers	1	1
	Rangers	Choppers	4	2
	Choppers	Ducks	0	5

Relations in Math

- A **domain** is a set of values. *e.g. the set of non-negative integers*
↳ finite or infinite
- Suppose $D_1, D_2, \dots, D_n$ are domains.
 - The **Cartesian product** $D_1 \times D_2 \times \dots \times D_n$ is the set of all tuples $\langle d_1, d_2, \dots, d_n \rangle$ such that $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$.
 - I.e., every combination of a value from D_1 , a value from D_2 etc.
- A **(mathematical) relation** on $D_1, D_2, \dots, D_n$ is a subset of the Cartesian product.

Examples of three relations on D_1 , D_2 and D_3

$D_1 = \{p, q, r, s\}$, $D_2 = \{1, 2, 3\}$ and $D_3 = \{100, 200\}$.

✓ $\{\}$

✓ $\{ \langle p, 1, 100 \rangle, \langle p, 1, 200 \rangle, \langle p, 2, 100 \rangle, \langle p, 2, 200 \rangle, \langle p, 2, 100 \rangle, \langle p, 3, 200 \rangle, \langle q, 1, 100 \rangle, \langle q, 1, 200 \rangle, \langle q, 2, 100 \rangle, \langle q, 2, 200 \rangle, \langle q, 2, 100 \rangle, \langle q, 3, 200 \rangle, \langle r, 1, 100 \rangle, \langle r, 1, 200 \rangle, \langle r, 2, 100 \rangle, \langle r, 2, 200 \rangle, \langle r, 2, 100 \rangle, \langle p, 3, 200 \rangle, \langle s, 1, 100 \rangle, \langle s, 1, 200 \rangle, \langle s, 2, 100 \rangle, \langle s, 2, 200 \rangle, \langle s, 2, 100 \rangle, \langle s, 3, 200 \rangle \}$

denotes a tuple

✓ $\{ \langle q, 2, 100 \rangle, \langle s, 3, 200 \rangle, \langle p, 1, 200 \rangle \}$

Question

Suppose:

$D_1 = \{p, q, r, s\}$, $D_2 = \{1, 2, 3\}$ and $D_3 = \{100, 200\}$.

What is the greatest possible number of tuples in a relation on D_1 , D_2 and D_3 ?

A. 4

B. 3

C. 2

D. 9

(E) 24

$$|D_1| \times |D_2| \times |D_3|$$

$$4 \times 3 \times 2$$

$$= 24$$

A database table is a relation

- Here's a table:

homeTeam	awayTeam	homeGoals	awayGoals
Rangers	Ducks	3	0
Ducks	Choppers	1	1
Rangers	Choppers	4	2
Choppers	Ducks	0	5

- Here it is as a relation:

Domain: Teams

Domain: non
negative numbers

{
 <Rangers, Ducks, 3, 0>,
 <Ducks, Choppers, 1, 1>,
 <Rangers, Choppers, 4, 2>,
 <Choppers, Ducks, 0, 5>

Terminology

Teams

homeTeam	awayTeam	homeGoals	awayGoals
Rangers	Ducks	3	0
Ducks	Choppers	1	1
Rangers	Choppers	4	2
Choppers	Ducks	0	5

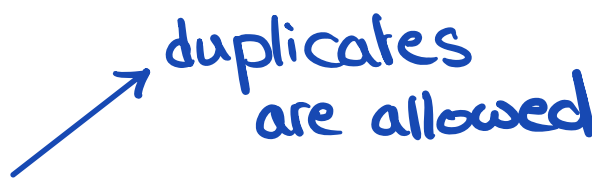
arity: 4

cardinality:
4

- **relation** (table)
- **attribute** (column)
- **tuple** (row)
- **@**arity of a relation: number of **@**attributes
- **cardinality** of a relation: number of tuples

```
{  
  <Rangers, Ducks, 3, 0>,  
  <Ducks, Choppers, 1, 1>,  
  <Rangers, Choppers, 4, 2>,  
  <Choppers, Ducks, 0, 5>  
}
```


Relations are sets

- A relation is a *set* of tuples, which means:
 - there can be no duplicate tuples
 - order of the tuples doesn't matter
- In another model, relations are bags — a generalization of sets that allows duplicates.  duplicates are allowed
- Commercial DBMSs use this model.
- But for now, we will stick with relations as sets.

Relation schemas vs instances

Schema

- The structure of a relation.
- Example, using an informal description:
 - Teams have 3 attributes: name, home field, coach.
 - No two teams can have the same name. → *constraint*
- Same example, but using formal notation:
 - Teams(name, homeField, coach)*

Instance

- The content of a relation.
- Notation:
 - We usually write down the data in a table.

Schemas vs instances: one more time

Schema

Teams(name, homeField, coach)

Instance

name	homeField	coach
Rangers	Runnymede CI	Tarvo Sinervo
Ducks	Humber Public	Tracy Zheng
Choppers	High Park	Ammar Jalali
Crullers	WTCS	Anna Liu

Database schemas and instances

- **Database schema:** a set of relation schemas.
- **Database instance:** a set of relation instances.

Change in a database

- Instances change constantly.
- Schemas (should) change rarely.
- Databases usually store the current version of the data.
- Databases that record the history are called *temporal* databases.

A schema for our league data

Teams(name, homeField, coach)

Games(homeTeam, awayTeam, homeGoals, awayGoals)

- Can there be >1 team with the same name?
 - Didn't happen in our dataset, but could it?
 - That's up to the league, not us!

* Luck vs. enforced

* yes!

A schema for our league data

Teams(name, homeField, coach)

Games(homeTeam, awayTeam, homeGoals, awayGoals)

- Suppose the league wants to allow
 - multiple teams with same name
 - multiple teams with same home field
- The schema allows it: nothing says otherwise.
- But what if the league wants to disallow
 - multiple teams with the same name *and* home field?

Keys

1 or more attributes

A **key** for a relation is a set of attributes $a_1, a_2, \dots, a_n$ such that:

uniqueness

$R(\underline{A}, B, C)$
X a b_1 c_1
 b_2 c_2

- ① Their combined values must be unique (i.e., this is a known fact about the domain).

Formally, $\nexists$ tuples t_1 and t_2 such that

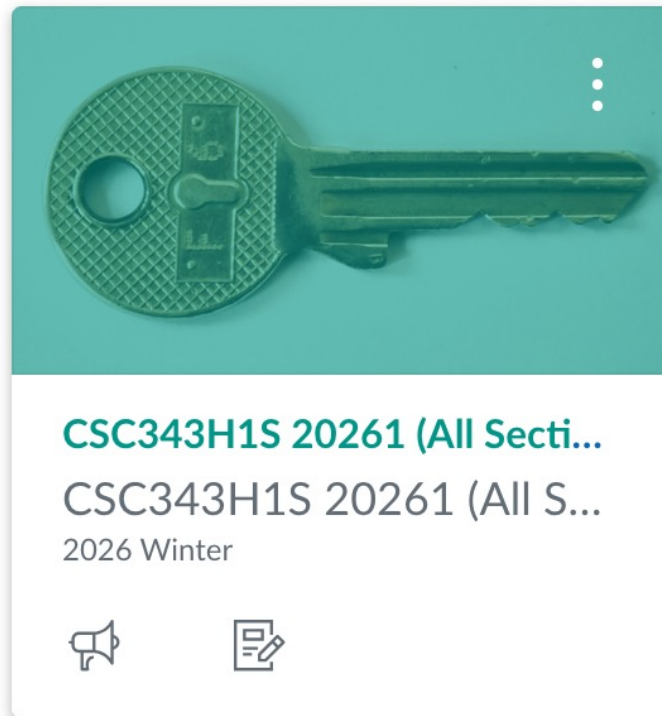
$$(t_1.a_1 = t_2.a_1) \wedge (t_1.a_2 = t_2.a_2) \wedge \dots \wedge (t_1.a_n = t_2.a_n)$$

- ② No subset of $a_1, a_2, \dots, a_n$ has this property.
minimality

Declaring a key constrains the data

- How to disallow multiple teams with the same name *and* home field?
 - Declare that name and homeField together are a key for the Teams relation.
- Notation for this: underline.
Teams(name, homeField, coach)
- Meaning: $\nexists$ tuples t_1 and t_2 such that
 $(t_1.name = t_2.name) \wedge$
 $(t_1.homeField = t_2.homeField)$

Why are they called “keys”?



Superkeys

- Any superset of a key is called a **superkey**.
 - ① uniqueness ✓
 - ② minimality
 - not necessary
- They are useful for database theory.
In practice, we only declare keys not superkeys.
- Can you identify some superkeys in the Teams relation?

Teams(name, homeField, coach)

Superkeys: { name, homefield, coach }
{ name, homefield }

Teams (name, homeField, coach)

Rangers	HighPark	Ammar
Rangers	<u>?</u>	Trawo

anything that's not
high park

Does { name, coach } have to be unique?

Teams (name, homeField, coach)

Rangers	HighPark	Ammar
Rangers	LowPark	Ammar

No! the key doesn't enforce { name, coach } to be unique.

Teams (name, homefield, coach, assistant)

keys: { name, homefield }

superkeys: { name, homefield }

{ name, homefield, coach }

{ name, homefield, assistant }

{ name, homefield, coach, assistant }

Teams (name, homeField, coach)

{	Rangers	High Park	Ammar
}	Rangers	High Park	Ammar



These are the same tuple (because we are dealing with sets).

Note: a key is also a superkey

Example

Understanding Keys and superkeys

Consider this mystery domain - Let's say we have a relation R on attributes a, b, c, d and our domain expert told us the following:

- $\{a, b, c\}$ are not allowed to repeat.
- Not even $\{b, c\}$ can repeat.
- b alone can repeat
- c alone can repeat
- a alone can repeat

what is the key in R ?

Can you identify the superkey(s) in R ?

R (a, b, c, d)

① X

1	2	3	4
1	2	3	14
5	6	7	8
15	6	7	18
2	4	6	10
12	4	16	100
120	14	16	200
120	17	18	19

②

③ ✓

④

⑤ ✓

The constraints according to the domain expert:

- ① - {a, b, c} are not allowed to repeat.
- ② - Not even {b, c} can repeat.
- ③ - b alone can repeat
- ④ - c alone can repeat
- ⑤ - a alone can repeat

① unique

② minimal

{a} X

{b} X

{c} X

{b, c} ✓

{a, b, c} ✓

not the key

not the key

not the key

key

superkey

✓

X

Concrete example

Course(dept, number, name, breadth)

One tuple might be

<“csc”, “343”, “Introduction to Databases”, True>

Suppose we know that no two tuples can have the same value for dept and number.

1. Is {dept, number} a key for this relation?

only if it is minimal — for most T this is True

2. How would we declare that it is a key?

see above!

Concrete example

Course(dept, number, name, breadth)

One tuple might be

<“csc”, “343”, “Introduction to Databases”, True>

Suppose we know that no two tuples can have the same value for dept and number.

3. Make an instance of Course that violates this constraint

a different courses with
dept = 'csc' } number = 343

4. Is {dept, number, name} a key?

No! a Superkey

Concrete example

Course(dept, number, name, breadth)

One tuple might be

<“csc”, “343”, “Introduction to Databases”, True>

Suppose we know that no two tuples can have the same value for dept and number.

5. Does every relation have a key?

yes! using set notation all
attributes form a key by default

$R(a, b, c)$
~~1 2 3~~
~~1 2 3~~

6. Can a relation have more than one key?

yes!

An example of a relation with 2 different
keys:

Student (sid, name, email)

The difference between multi-attribute keys
 e.g. {dept, number} and having two separate
 keys: {dept} {number}:

Course (dept, number, name, breadth)

	CSC	343	Intro DBs	T
X	CSC	343	DBS II	T
✓	CSC	108	Intro to CS	T
✓	Bio	343	Intro to Neuro an	T

Course (dept, number, name, breadth)

Silly!

X	CSC	343	DBs	T
	CSC	108	Intro	T
X	CSC	343	DBs	T
	Bio	343	Neuro	T

Keys vs superkeys: one more time

- **Key**: a minimal set of attributes such that no two tuples can have the same values on all of these attributes.
- **Superkey**: a superset of some key (not necessarily a proper superset). $R(\underline{a}, b, c)$

Coincidence vs key

- If a set of attributes is a key for a relation:
 - It does not mean merely that there are no duplicates in a particular instance of the relation
 - It means that in principle there *cannot* be any.
 - Only a domain expert can determine that.
- Often we invent an attribute to ensure all tuples will be unique.
 - This predates databases.
 - E.g., SIN, ISBN number, VIN.
- A key defines a kind of integrity constraint.

“Primary key”??

- Some of you have heard the term primary key.
- This is specific to SQL and we’ll learn about it later.

Example: Movies schema handout, Q1-6

References between relations

- Relations often refer to each other.
- Example:
In the Roles relation, the tuple about Han Solo needs to say he is played by Ford.
- Rather than repeat information already in the Artists table, we store Ford's key.
- If aID is a key for Artists, does that mean a particular aID can appear only once in Roles??

Not necessarily

Foreign keys

- The referring attribute is called a **foreign key** because it refers to an attribute that is a key in another table.
- This gives us a way to refer to a **single tuple** in that relation.
- A foreign key may need to have several attributes.

Declaring foreign keys: new notation

↪ 1 or more attrs

- First, a bit of notation: $R[A]$
 - R is a relation and A is a list of attributes in R .
 - $R[A]$ is the set of all tuples from R , but with only the attributes in list A .

Artists

aID	aName	nat
1	Nicholson	American
2	Ford	American
3	Stone	British
4	Fisher	American

Artists[aName, nat]

aName	nat
Nicholson	American
Ford	American
Stone	British
Fisher	American

Artists[nat]

nat
American
British



because
of set
notation

no duplicates

result: a relation

Or to say it mathematically ...

Artists = {
 <1, Nicholson, American>,
 <2, Ford, American>,
 <3, Stone, British>,
 <4, Fisher, American> }

Artists[aName, nat] = {
 <Nicholson, American>,
 <Ford, American>,
 <Stone, British>,
 <Fisher, American> }

Artists[nat] = {<American>, <British>}

Declaring foreign keys

- We declare foreign key constraints this way:

$$R_1[X] \subseteq R_2[Y] \quad \text{1 or more attrs}$$

- X and Y may be lists of attributes, of same arity
- Y must be a key in R_2

- Example:

$$\text{Roles}[\text{mID}] \subseteq \text{Movies}[\text{mID}]$$

Or to say it mathematically ...

Roles = {

<1, 1, Jack Torrance>, <3, 1, Harry Sanborn>,
<1, 3, Delbert Grady>, <5, 2, Han Solo>,
<6, 2, Bob Falfa>, <5, 4, Princess Leia Organa>}

Roles[mID] = {<1>, <3>, <5>, <6>}

Movies[mID] = {<1>, <2>, <3>, <4>, <5>, <6>, <7>}

Is Roles[mID] $\subseteq$ Movies[mID]? *yes!*

Example: Movies schema, Q7-11

Referential integrity constraints

- These $R_1[X] \subseteq R_2[Y]$ relationships are a kind of **referential integrity constraint**.
- Not all referential integrity constraints are foreign key constraints.
 - $R_1[X] \subseteq R_2[Y]$ is a foreign key constraint iff **Y is a key for relation R_2** .
- For example, we could say
 $\text{Artists}[\text{aID}] \subseteq \text{Roles}[\text{aID}]$
Side effect: we may not be referring to a unique tuple.
an artist needs to star in at least 1 movie to be an artist

Roles (mID, aID, character)

Designing a schema

- Mapping from the real world to a relational schema is surprisingly challenging and interesting.
- There are always many possible schemas.
- Two important goals:
 - Represent the data well.
For example, avoid constraints that prevent expressing things that occur in the domain.
 - Avoid redundancy.
- Later, we'll learn some elegant theory that provides sound principles for good design.

Learning outcome self-check

Are you able to do these things?

- Read and interpret a schema written in relational notation
- Understand the difference between schema and instance
- Identify instances that do and don't violate it
- Know how to use terminology, e.g., tuple, arity
- Explain and use the concepts key, superkey, and foreign key

What's next

- We will learn how to use SQL to
 - define a database's structure,
 - put data in it, and
 - write queries on it.
- First we'll learn how to write queries in relational algebra.
 - Relational algebra is the foundation for SQL.
 - Other important concepts, like query optimization, are defined in terms of RA.